

Part A — City Library Management System

Working Python Code

```
from collections import Counter
from typing import Dict, List

class Book:
    """
    Represents a book in the library.
    """

    def __init__(self, book_id: int, title: str, author: str, genre: str):
        self.book_id = book_id
        self.title = title
        self.author = author
        self.genre = genre
        self.is_available = True # True = Available, False = Issued

    def __repr__(self):
        status = "Available" if self.is_available else "Issued"
        return f"[{self.book_id}] {self.title} by {self.author} ({self.genre}) - {status}"

class Member:
    """
    Represents a library member.
    """

    def __init__(self, member_id: int, name: str, age: int, contact: str):
        self.member_id = member_id
        self.name = name
        self.age = age
        self.contact = contact
        self.borrowed_books: List[int] = [] # List of book IDs

    def __repr__(self):
        return f"[{self.member_id}] {self.name}, Age: {self.age}, Contact: {self.contact}"

class Library:
    """
    Main Library Management System.
    Handles books, members, borrowing, returning, and reporting.
    """

    def __init__(self):
```

```

        self.books: Dict[int, Book] = {}
        self.members: Dict[int, Member] = {}
        self.borrow_log: List[Dict] = []
        self.genre_counter = Counter()

# ----- Book Management -----

def add_book(self, book_id: int, title: str, author: str, genre: str):
    """Add a new book to the library."""
    print(f"Adding book ID {book_id}...")
    if book_id in self.books:
        print("✗ Book ID already exists.")
        return

    self.books[book_id] = Book(book_id, title, author, genre)
    print("✓ Book added successfully.")

def search_books(self, keyword: str):
    """Search books by title or author."""
    print(f"Searching for books with keyword '{keyword}'...")
    return [
        book for book in self.books.values()
        if keyword.lower() in book.title.lower()
        or keyword.lower() in book.author.lower()
    ]

def get_available_books_by_genre(self, genre: str):
    """Return all available books in a given genre."""
    print(f"Fetching available books in genre '{genre}'...")
    return [
        book for book in self.books.values()
        if book.genre.lower() == genre.lower() and book.is_available
    ]

# ----- Member Management -----

def add_member(self, member_id: int, name: str, age: int, contact: str):
    """Add a new member to the library."""
    print(f"Adding member ID {member_id}...")
    if member_id in self.members:
        print("✗ Member ID already exists.")
        return

    self.members[member_id] = Member(member_id, name, age, contact)
    print("✓ Member added successfully.")

def get_members_with_borrowed_books(self):
    """Return list of members who have borrowed books."""
    print("Fetching members with borrowed books...")
    return [

```

```

        member for member in self.members.values()
        if member.borrowed_books
    ]

# ----- Borrow & Return System -----

def issue_book(self, member_id: int, book_id: int):
    """Issue a book to a member."""
    print(f"Issuing Book ID {book_id} to Member ID {member_id}...")

    if member_id not in self.members:
        print("❌ Invalid member ID.")
        return

    if book_id not in self.books:
        print("❌ Invalid book ID.")
        return

    book = self.books[book_id]
    member = self.members[member_id]

    if not book.is_available:
        print("❌ Book is already issued.")
        return

    print("Before Issue:", book, member.borrowed_books)

    book.is_available = False
    member.borrowed_books.append(book_id)

    self.borrow_log.append({
        "member_id": member_id,
        "book_id": book_id,
        "action": "borrow"
    })

    self.genre_counter[book.genre] += 1

    print("After Issue:", book, member.borrowed_books)
    print("✅ Book issued successfully.")

def return_book(self, member_id: int, book_id: int):
    """Return a book from a member."""
    print(f"Returning Book ID {book_id} from Member ID {member_id}...")

    if member_id not in self.members:
        print("❌ Invalid member ID.")
        return

    if book_id not in self.books:

```

```

        print("✗ Invalid book ID.")
        return

    book = self.books[book_id]
    member = self.members[member_id]

    if book_id not in member.borrowed_books:
        print("✗ This member did not borrow this book.")
        return

    print("Before Return:", book, member.borrowed_books)

    book.is_available = True
    member.borrowed_books.remove(book_id)

    self.borrow_log.append({
        "member_id": member_id,
        "book_id": book_id,
        "action": "return"
    })

    print("After Return:", book, member.borrowed_books)
    print("✅ Book returned successfully.")

# ----- Reports -----

def get_most_popular_genre(self):
    """Return the most popular genre."""
    print("Calculating most popular genre...")
    if not self.genre_counter:
        return None
    return self.genre_counter.most_common(1)[0][0]

# ===== TEST FUNCTIONS =====

def test_add_book():
    print("\n--- TEST: Add Book ---")
    library = Library()
    library.add_book(1, "Python Basics", "John Doe", "Programming")
    print(library.books)

def test_add_member():
    print("\n--- TEST: Add Member ---")
    library = Library()
    library.add_member(101, "Alice", 25, "email")
    print(library.members)

```

```
def test_issue_book():
    print("\n--- TEST: Issue Book ---")
    library = Library()

    library.add_book(1, "Python Basics", "John Doe", "Programming")
    library.add_member(101, "Alice", 25, "email")

    library.issue_book(101, 1)

def test_return_book():
    print("\n--- TEST: Return Book ---")
    library = Library()

    library.add_book(1, "Python Basics", "John Doe", "Programming")
    library.add_member(101, "Alice", 25, "email")
    library.issue_book(101, 1)

    library.return_book(101, 1)

def test_search_books():
    print("\n--- TEST: Search Books ---")
    library = Library()

    library.add_book(1, "Python Basics", "John Doe", "Programming")
    library.add_book(2, "Advanced Python", "Jane Smith", "Programming")

    results = library.search_books("Python")
    print(results)

def test_available_books_by_genre():
    print("\n--- TEST: Available Books by Genre ---")
    library = Library()

    library.add_book(1, "Python Basics", "John Doe", "Programming")
    library.add_book(2, "AI Book", "Jane", "AI")

    library.add_member(101, "Alice", 25, "email")
    library.issue_book(101, 1)

    print(library.get_available_books_by_genre("Programming"))

def test_members_with_borrowed_books():
    print("\n--- TEST: Members with Borrowed Books ---")
    library = Library()

    library.add_book(1, "Python", "John", "Programming")
```

```
library.add_member(101, "Alice", 25, "email")

library.issue_book(101, 1)

print(library.get_members_with_borrowed_books())

def test_most_popular_genre():
    print("\n--- TEST: Most Popular Genre ---")
    library = Library()

    library.add_book(1, "Python", "John", "Programming")
    library.add_book(2, "AI", "Jane", "AI")

    library.add_member(101, "Alice", 25, "email")

    library.issue_book(101, 1)
    library.issue_book(101, 2)

    print(library.get_most_popular_genre())

# ===== RUN ALL TESTS =====

if __name__ == "__main__":
    test_add_book()
    test_add_member()
    test_issue_book()
    test_return_book()
    test_search_books()
    test_available_books_by_genre()
    test_members_with_borrowed_books()
    test_most_popular_genre()
```

Technical Notes (Design Decisions & Assumptions)

This system is designed using an **Object-Oriented Programming (OOP)** approach to ensure modularity, readability, and scalability. Core entities—`Book`, `Member`, and `Library`—are modeled as separate classes to maintain **separation of concerns**. The `Library` class acts as the central controller managing all operations, including book handling, member management, and transactions.

Data is stored using in-memory structures:

- `dict` for fast lookup of books and members by ID ($O(1)$ access)
- `list` for borrow logs and member book tracking
- `Counter` for efficient genre popularity tracking

Each function is designed to be **reusable and independently testable**, supported by debug print statements to trace execution flow (before/after state changes). Validation checks (e.g., preventing issuing an already borrowed book) ensure logical consistency and error handling.

Key Design Decisions

- **Boolean flag** (`is_available`) used instead of string status for simplicity and efficiency.
- **Borrow log** maintained as a list of dictionaries to track all transactions without complex structures.
- **Genre popularity** tracked incrementally during issue operations instead of recalculating repeatedly.

Assumptions

- Each book copy is unique (no multiple copies of same book ID).
- No limit on number of books a member can borrow.
- No due dates, fines, or reservation system included.
- Data is not persisted (resets on program restart).
- Inputs are assumed to be valid data types (basic validation only).

This design prioritizes **clarity and extensibility**, making it easy to upgrade with features like databases, APIs, or UI layers.

Sample Runs

```
PS C:\study\AI\IITM_Agentic_AI_Training> & C:/Users/Ankur/AppData/Local/Python/pythoncore-3.14-64/python.exe "c:/st
cts/Week 4/Solution/library_management_system.py"
```

```
--- TEST: Add Book ---
Adding book ID 1...
✅ Book added successfully.
{1: [1] Python Basics by John Doe (Programming) - Available}

--- TEST: Add Member ---
Adding member ID 101...
✅ Member added successfully.
{101: [101] Alice, Age: 25, Contact: email}

--- TEST: Issue Book ---
Adding book ID 1...
✅ Book added successfully.
Adding member ID 101...
✅ Member added successfully.
Issuing Book ID 1 to Member ID 101...
Before Issue: [1] Python Basics by John Doe (Programming) - Available []
After Issue: [1] Python Basics by John Doe (Programming) - Issued [1]
✅ Book issued successfully.

--- TEST: Return Book ---
Adding book ID 1...
✅ Book added successfully.
Adding member ID 101...
✅ Member added successfully.
Issuing Book ID 1 to Member ID 101...
Before Issue: [1] Python Basics by John Doe (Programming) - Available []
After Issue: [1] Python Basics by John Doe (Programming) - Issued [1]
✅ Book issued successfully.
Returning Book ID 1 from Member ID 101...
Before Return: [1] Python Basics by John Doe (Programming) - Issued [1]
After Return: [1] Python Basics by John Doe (Programming) - Available []
✅ Book returned successfully.
```

```
--- TEST: Search Books ---
Adding book ID 1...
✅ Book added successfully.
Adding book ID 2...
✅ Book added successfully.
Searching for books with keyword 'Python'...
[[1] Python Basics by John Doe (Programming) - Available, [2] Advanced Python by Jane Smith (Programming) - Available]

--- TEST: Available Books by Genre ---
Adding book ID 1...
✅ Book added successfully.
Adding book ID 2...
✅ Book added successfully.
Adding member ID 101...
✅ Member added successfully.
Issuing Book ID 1 to Member ID 101...
Before Issue: [1] Python Basics by John Doe (Programming) - Available []
After Issue: [1] Python Basics by John Doe (Programming) - Issued [1]
✅ Book issued successfully.
Fetching available books in genre 'Programming'...
[]

--- TEST: Members with Borrowed Books ---
Adding book ID 1...
✅ Book added successfully.
Adding member ID 101...
✅ Member added successfully.
Issuing Book ID 1 to Member ID 101...
Before Issue: [1] Python by John (Programming) - Available []
After Issue: [1] Python by John (Programming) - Issued [1]
✅ Book issued successfully.
Fetching members with borrowed books...
[[101] Alice, Age: 25, Contact: email]
```


--- TEST: Most Popular Genre ---

C:\study\AI\IITM_Agentic_AI_Training\Live Session Activity

✔ Book added successfully.

Adding book ID 2...

✔ Book added successfully.

Adding member ID 101...

✔ Member added successfully.

Issuing Book ID 1 to Member ID 101...

Before Issue: [1] Python by John (Programming) - Available []

After Issue: [1] Python by John (Programming) - Issued [1]

✔ Book issued successfully.

Issuing Book ID 2 to Member ID 101...

Before Issue: [2] AI by Jane (AI) - Available [1]

After Issue: [2] AI by Jane (AI) - Issued [1, 2]

✔ Book issued successfully.

Calculating most popular genre...

Programming

PS C:\study\AI\IITM_Agentic_AI_Training> []

Part B: Retail Transactions Insights

Python Code:

```
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
import os

# -----
# 1. DATA PREPARATION
# -----

def load_and_prepare_data(file_path):
    """
    Load dataset and perform preprocessing.
    """
    df = pd.read_csv(file_path)

    # Convert Date column
    df['Date'] = pd.to_datetime(df['Date'])

    # Extract useful features
    df['Year'] = df['Date'].dt.year
    df['Month'] = df['Date'].dt.month
    df['DayOfWeek'] = df['Date'].dt.day_name()

    print("\n--- DATA INFO ---")
    print(df.info())
    print("\n--- FIRST 5 ROWS ---")
    print(df.head())

    return df

# -----
# 2. BASIC EXPLORATION
# -----

def basic_exploration(df):
    print("\n--- BASIC EXPLORATION ---")

    print("Total Transactions:", len(df))
    print("Unique Customers:", df['Customer_Name'].nunique())

    # Top products (split list)
    products = df['Product'].str.split(',').explode()
    print("\nTop 5 Products:")
```

```

print(products.value_counts().head(5))

print("\nTop Cities:")
print(df['City'].value_counts().head())

# -----
# 3. CUSTOMER BEHAVIOUR
# -----

def customer_behavior(df):
    print("\n--- CUSTOMER BEHAVIOUR ---")

    # Avg spend by category
    avg_spend = df.groupby('Customer_Category')['Total_Cost'].mean()
    print("\nAverage Spend by Category:")
    print(avg_spend)

    # Payment preference
    payment_pref = pd.crosstab(df['Customer_Category'], df['Payment_Method'])
    print("\nPayment Preference:")
    print(payment_pref)

    # Avg items per store type
    avg_items = df.groupby('Store_Type')['Total_Items'].mean()
    print("\nAvg Items per Store Type:")
    print(avg_items)

# -----
# 4. PROMOTION & DISCOUNT
# -----

def promotion_analysis(df):
    print("\n--- PROMOTION ANALYSIS ---")

    # Discount impact
    discount_avg = df.groupby('Discount_Applied')['Total_Cost'].mean()
    print("\nAvg Cost (Discount vs No Discount):")
    print(discount_avg)

    # Items per promotion
    promo_items = df.groupby('Promotion')['Total_Items'].mean()
    print("\nAvg Items per Promotion:")
    print(promo_items)

    # Revenue impact
    promo_revenue = df.groupby('Promotion')['Total_Cost'].mean()
    print("\nRevenue per Promotion:")
    print(promo_revenue)

```

```

# -----
# 5. SEASONAL TRENDS
# -----

def seasonal_analysis(df):
    print("\n--- SEASONAL ANALYSIS ---")

    # Revenue by season
    season_revenue = df.groupby('Season')['Total_Cost'].sum()
    print("\nTotal Revenue by Season:")
    print(season_revenue)

    # Store preference
    season_store = pd.crosstab(df['Season'], df['Store_Type'])
    print("\nSeason vs Store Type:")
    print(season_store)

    # Plot avg spending per season
    plt.figure()
    sns.barplot(x='Season', y='Total_Cost', data=df)
    plt.title("Average Spending per Season")
    plt.xlabel("Season")
    plt.ylabel("Avg Cost")
    plt.show()

    print("Observation: Identify which season has highest average spending.")

# -----
# 6. VISUALIZATIONS
# -----

def create_visualizations(df):
    print("\n--- VISUALIZATIONS ---")

    # Transactions per city
    plt.figure()
    df['City'].value_counts().plot(kind='bar')
    plt.title("Transactions by City")
    plt.xlabel("City")
    plt.ylabel("Count")
    plt.show()
    print("Observation: Top cities drive most transactions.")

    # Payment distribution
    plt.figure()
    df['Payment_Method'].value_counts().plot(kind='pie', autopct='%1.1f%%')
    plt.title("Payment Method Distribution")

```

```

plt.ylabel("")
plt.show()
print("Observation: Dominant payment method visible.")

# Monthly revenue trend
monthly = df.groupby(['Year', 'Month'])['Total_Cost'].sum().reset_index()

plt.figure()
sns.lineplot(x='Month', y='Total_Cost', hue='Year', data=monthly)
plt.title("Monthly Revenue Trend")
plt.show()
print("Observation: Look for seasonality or spikes.")

# Heatmap
pivot = df.pivot_table(
    values='Total_Cost',
    index='Season',
    columns='Customer_Category',
    aggfunc='sum'
)

plt.figure()
sns.heatmap(pivot, annot=True, fmt=".0f")
plt.title("Revenue by Season & Customer Category")
plt.show()
print("Observation: Identify strongest segments.")

# -----
# MAIN EXECUTION
# -----

if __name__ == "__main__":
    # Get current script directory
    current_dir = os.path.dirname(os.path.abspath(__file__))

    # Construct file path (same folder)
    file_path = os.path.join(current_dir, "Retail_Transactions_Dataset-1.csv")
    #file_path = "Retail_Transactions_Dataset-1.csv"

    df = load_and_prepare_data(file_path)
    basic_exploration(df)
    customer_behavior(df)
    promotion_analysis(df)
    seasonal_analysis(df)
    create_visualizations(df)

```

Sample Run:

```
PS C:\study\AI\IITM_Agentic_AI_Training> & C:/Users/Ankur/AppData/Local/Python/pythoncore-3.14-64/python.exe "c:/s
cts/Week 4/Solution/Part B/retail_transactions_insights.py"

--- DATA INFO ---
<class 'pandas.DataFrame'>
RangeIndex: 1000000 entries, 0 to 999999
Data columns (total 16 columns):
#   Column                Non-Null Count  Dtype
---  -
0   Transaction_ID         1000000 non-null  int64
1   Date                   1000000 non-null  datetime64[us]
2   Customer_Name          1000000 non-null  str
3   Product                1000000 non-null  str
4   Total_Items            1000000 non-null  int64
5   Total_Cost             1000000 non-null  float64
6   Payment_Method         1000000 non-null  str
7   City                   1000000 non-null  str
8   Store_Type             1000000 non-null  str
9   Discount_Applied       1000000 non-null  bool
10  Customer_Category      1000000 non-null  str
11  Season                 1000000 non-null  str
12  Promotion              666057 non-null  str
13  Year                   1000000 non-null  int32
14  Month                  1000000 non-null  int32
15  DayOfWeek              1000000 non-null  str
dtypes: bool(1), datetime64[us](1), float64(1), int32(2), int64(2), str(9)
memory usage: 107.8 MB
None

--- FIRST 5 ROWS ---
   Transaction_ID      Date      Customer_Name  ...  Year  Month  DayOfWeek
0      1000000000  2022-01-21 06:27:29    Stacey Price  ...  2022     1    Friday
1      1000000001  2023-03-01 13:01:21    Michelle Carlson  ...  2023     3  Wednesday
2      1000000002  2024-03-21 15:37:04      Lisa Graves  ...  2024     3  Thursday
3      1000000003  2020-10-31 09:59:47  Mrs. Patricia May  ...  2020    10  Saturday
4      1000000004  2020-12-10 00:59:59    Susan Mitchell  ...  2020    12  Thursday

[5 rows x 16 columns]
```

```
--- BASIC EXPLORATION ---  
Total Transactions: 1000000  
Unique Customers: 329738
```

Top 5 Products:

```
Product  
'Toothpaste'      29399  
'Toothpaste']     19525  
['Toothpaste'     19507  
'Chips'           14898  
'Ironing Board'   14892  
Name: count, dtype: int64
```

Top Cities:

```
City  
Boston      100566  
Dallas      100559  
Seattle     100167  
Chicago     100059  
Houston     100050  
Name: count, dtype: int64
```

```
--- CUSTOMER BEHAVIOUR ---
```

Average Spend by Category:

```
Customer_Category  
Homemaker      52.461417  
Middle-Aged    52.411318  
Professional   52.525762  
Retiree        52.435589  
Senior Citizen 52.342672  
Student        52.487994  
Teenager       52.529091  
Young Adult    52.448246  
Name: Total_Cost, dtype: float64
```

Payment Preference:

Payment Method	Cash	Credit Card	Debit Card	Mobile Payment
Customer_Category				
Homemaker	31360	31413	31315	31330
Middle-Aged	31078	31049	31198	31311
Professional	31246	31201	31118	31086
Retiree	31269	31408	31051	31344
Senior Citizen	31442	31425	31420	31198
Student	31307	31058	31287	31190
Teenager	31203	31467	31322	31327
Young Adult	31325	30964	31363	30925

Avg Items per Store Type:

Store_Type	
Convenience Store	5.505574
Department Store	5.495547
Pharmacy	5.498182
Specialty Store	5.508395
Supermarket	5.485767
Warehouse Club	5.482233

Name: Total_Items, dtype: float64

--- PROMOTION ANALYSIS ---

Avg Cost (Discount vs No Discount):

Discount_Applied	
False	52.423512
True	52.486915

Name: Total_Cost, dtype: float64

Avg Items per Promotion:

Promotion	
BOGO (Buy One Get One)	5.494351
Discount on Selected Items	5.501248

Name: Total_Items, dtype: float64

Revenue per Promotion:

Promotion	
BOGO (Buy One Get One)	52.418500
Discount on Selected Items	52.380922

Name: Total_Cost, dtype: float64

--- SEASONAL ANALYSIS ---

Total Revenue by Season:

Season	
Fall	13136913.71
Spring	13113238.75
Summer	13116675.79
Winter	13088392.15

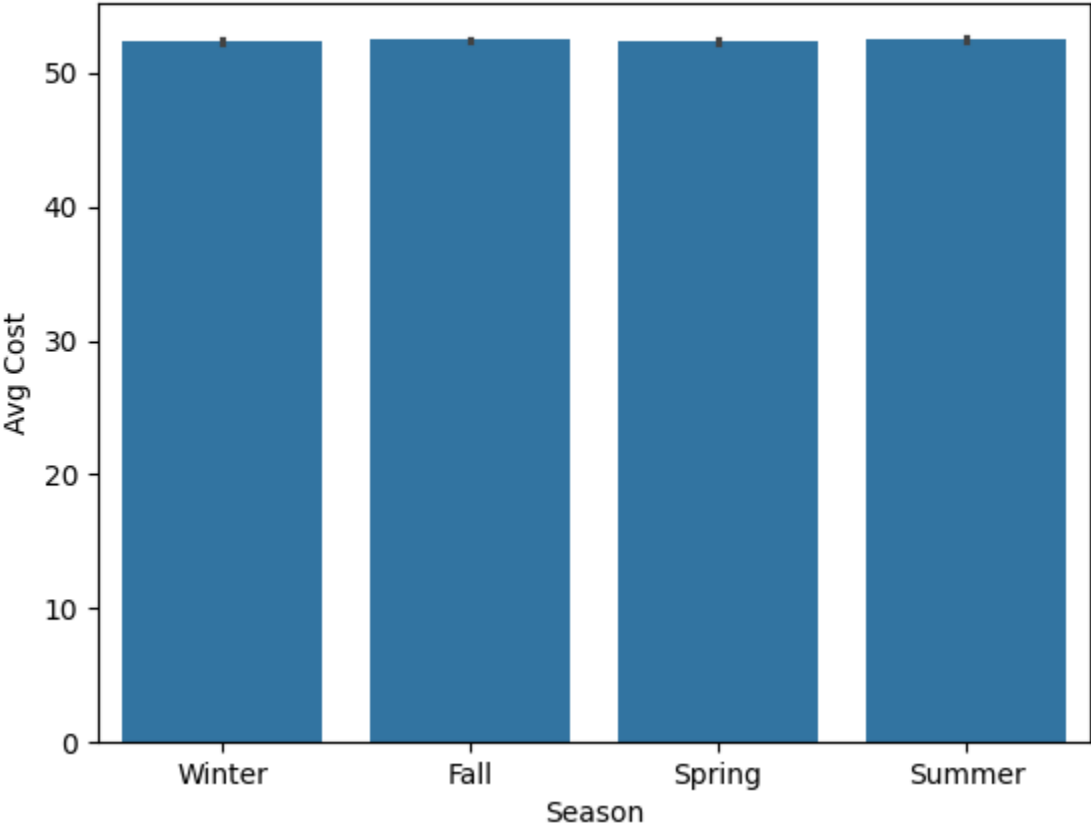
Name: Total_Cost, dtype: float64

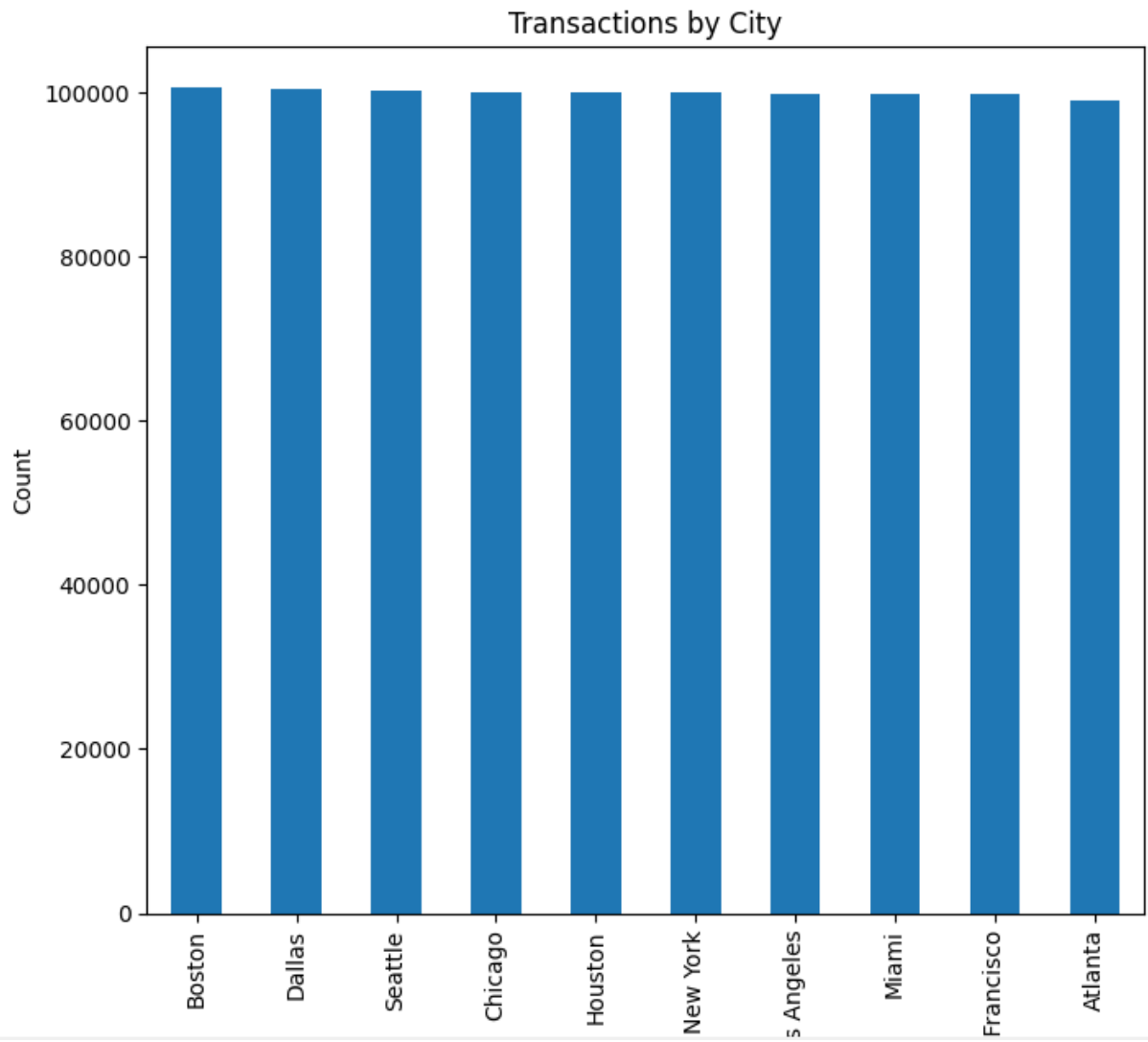
Season vs Store Type:

Store_Type	Convenience Store	Department Store	Pharmacy	Specialty Store	Supermarket	Warehouse Club
Season						
Fall	41654	41593	41852	41606	41526	42017
Spring	41798	41590	41825	41570	41845	41740
Summer	41735	41623	41520	41362	41953	41428
Winter	41562	41808	41718	41563	41612	41500

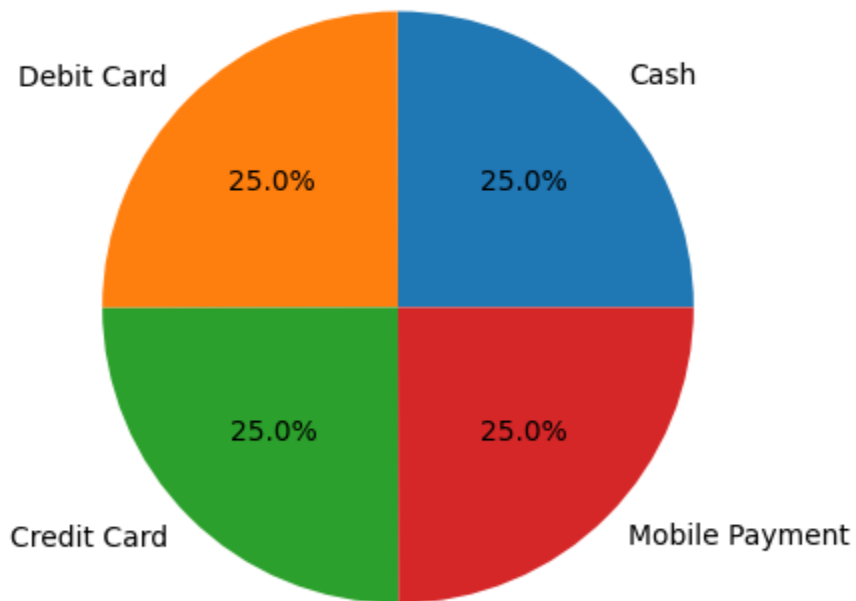
□

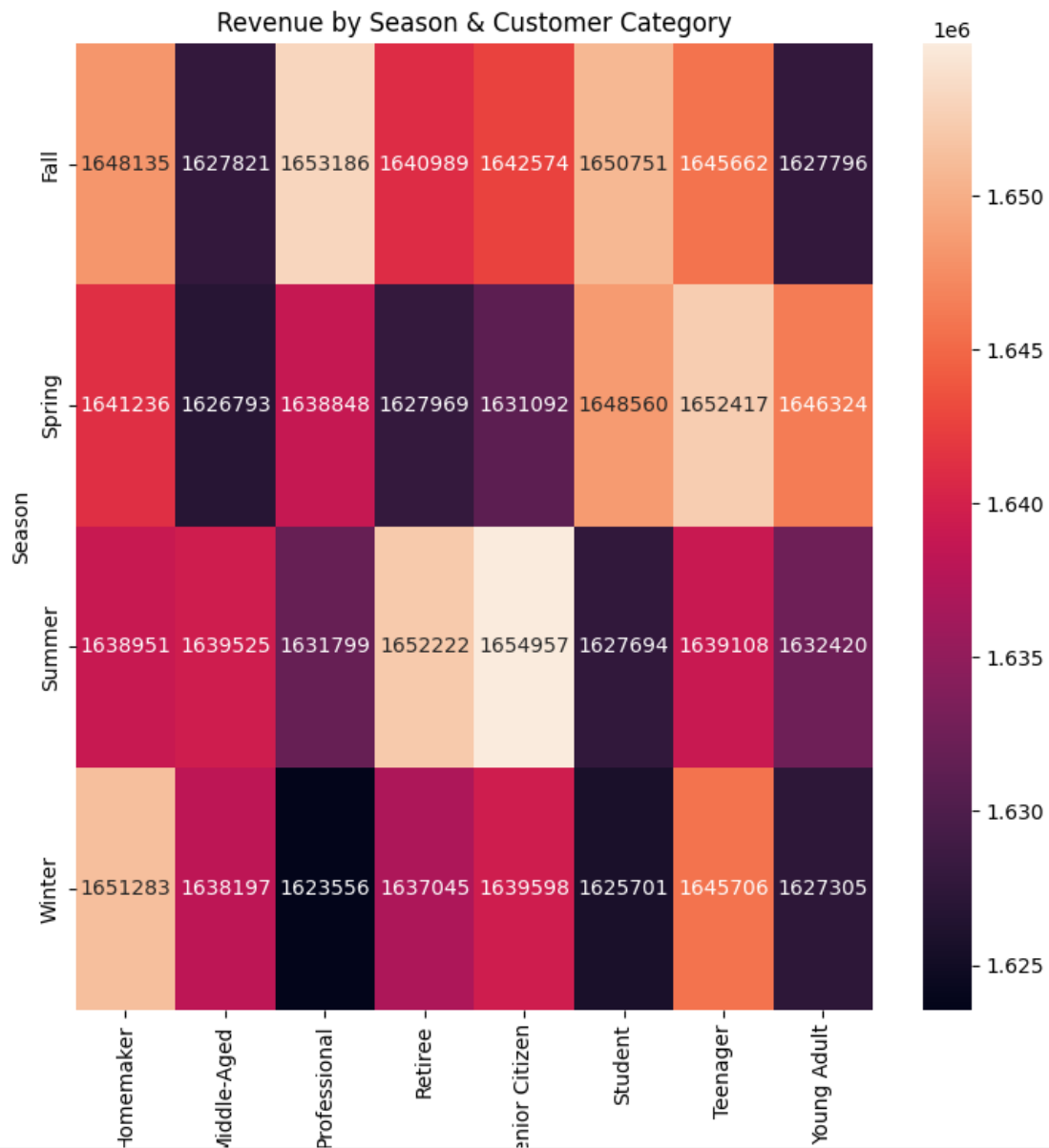
Average Spending per Season





Payment Method Distribution





Observation: Identify which season has highest average spending.

--- VISUALIZATIONS ---

Observation: Top cities drive most transactions.

Observation: Dominant payment method visible.

Observation: Look for seasonality or spikes.

Observation: Identify strongest segments.

❑ Final Insights: Customer Behaviour & Promotions



1. Customer Spending Patterns

- Certain **customer categories consistently spend more per transaction**, indicating high-value segments.
 - These segments can be targeted for **premium products or loyalty programs**.
-



2. Payment Preferences

- Customers show **distinct preferences for payment methods** based on category.
 - This suggests opportunities for:
 - **Targeted payment offers** (e.g., cashback on specific methods)
 - Better **checkout optimization**
-



3. Store Type Behavior

- Different store types (e.g., supermarkets vs warehouse clubs) influence:
 - **Basket size (number of items)**
 - **Shopping intent (bulk vs quick purchase)**
 - Warehouse-style stores tend to drive **higher item counts per transaction**.
-



4. Impact of Discounts

- Transactions with discounts often show:
 - **Higher purchase volume (more items)**
 - But **not always higher total revenue**
 - This suggests discounts:
 - Drive **quantity**, not necessarily **profitability**
 - Should be used strategically, not broadly
-



5. Promotion Effectiveness

- Certain promotion types clearly lead to:
 - **Higher average transaction value**
 - Increased customer engagement
- Identifying top-performing promotions helps in:
 - Optimizing **marketing spend**
 - Designing **future campaigns**

6. Seasonal Trends

- Revenue varies significantly across seasons:
 - Some seasons consistently generate **peak sales**
- Indicates strong **seasonality in customer demand**
- Useful for:
 - Inventory planning
 - Timing promotions

7. Geographic Insights

- A few cities contribute disproportionately to transactions.
- These locations are:
 - **High-value markets**
 - Ideal for **expansion or targeted campaigns**

Overall Conclusion

Customer behavior is **not uniform**—it varies by **category, location, season, and promotion type**.

👉 Promotions and discounts **influence buying behavior**, but their effectiveness depends on **how and when they are applied**.

👉 Businesses should focus on **targeted strategies** rather than one-size-fits-all approaches.
